JENKINS JOB TEMPLATE ENGINE

Automating Jenkins Job creation using Teamplates

PRESENTED BY: **ANWESHA SINGH (23FE10CSE00299)**

COURSE: **DEVOPS (CSE3253)**

SEMESTER: **VI**

PROJECT TYPE: **JENKINS & CI**

MANIPAL UNIVERSITY

PROBLEM STATEMENT

In real-world projects, multiple Jenkins jobs need to be created
Manual job configuration is:

- Time-consuming
- Repetitive
- Error-prone
- Lack of standardisation across pipelines

Difficult to manage pipelines

Problem:
How can we automate Jenkins job creation in a ~~scalable~~ and reusable way?

Need **Jobs as Code**

PROPOSED SOLUTION

- Build a JobTemplate Engine
- Automate Jenkins jobs using:
  YAML (configuration)
  Jinja2 (template engine)
  Python (orchestration)
  Jenkins REST API

Result: Fully automated pipeline creation
Automatically generate multiple Jenkins Jobs with predefined stages
Define once → Reuse everywhere → Automate everything

OBJECTIVE

- Automate Jenkins job creation
- Reduce manual configuration effort
- Improve CI/CD efficiency
- Enable reusable Job templates
- Introduce parameterised pipelines

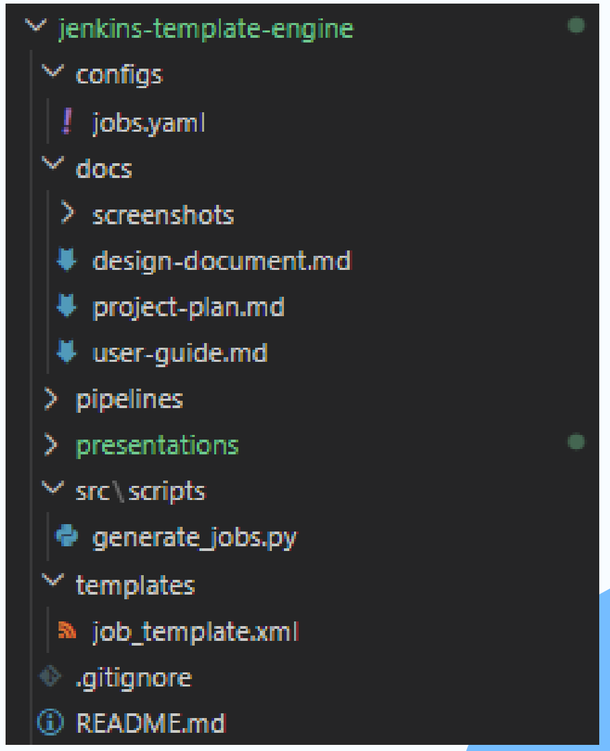
SYSTEM ARCHITECTURE

jobs.yaml → Python → XML → Jenkins → Execution

- Config layer → jobs.yaml
- Template layer → job_template.xml
- Logic/Script layer → generate_jobs.py
- Execution layer → Jenkins

Design Principle: Separation of concerns
YAML → Data
XML Template → Pipeline logic
Python → Execution logic
Advantage:

- Reusable pipeline
- Easy updates
- No duplication



YAML CONFIGURATION (JOBS.YAML)

jobs.yaml → Python → XML → Jenkins → Execution

The **Data layer**

jobs.yaml

jenkins-template-engine > configs > jobs.yaml > [] jobs

You, last week | 1 author (You)

```yaml
jobs:

  - name: job1
    description: Build Job 1
    repo_url: https://github.com/Anwesha-code/jenkins-job-template-engine.git
    branch: main
    environment: dev
    install_command: echo Installing dependencies
    build_command: echo Building Job 1
    test_command: echo Running tests

  - name: job2
    description: Build Job 2
    repo_url: https://github.com/Anwesha-code/jenkins-job-template-engine.git
    branch: main
    environment: staging
    install_command: echo Installing dependencies
    build_command: echo Building Job 2
    test_command: echo Running tests
```

- User defines jobs in jobs.yaml
- Stores **job specific parameters**

PYTHON SCRIPT (GENERATE_JOBS.PY)

jobs.yaml → Python → XML → Jenkins → Execution

Core automation engine

Responsibilities:

- Read YAML
- Load template
- Render XML
- Call Jenkins API

Fully automates job creation

jobs.yaml | job_template.xml | generate_jobs.py 3

jenkins-template-engine > src > scripts > generate_jobs.py > ...

You, last week | 1 author (You)

```python
import yaml
import requests
from jinja2 import Template

JENKINS_URL = "http://localhost:8080"
USERNAME = "AnweshaSingh"
API_TOKEN = "[redacted]"


def load_template():


def load_jobs():


def create_job(job_name, config_xml):


def main():
    template = load_template()
    jobs = load_jobs()

    for job in jobs:
        config_xml = template.render(**job)
        print("GENERATED XML")
        print(config_xml)
        create_job(job["name"], config_xml)


if __name__ == "__main__":
    main()
```

TEMPLATE (JOB_TEMPLATE.XML)

jobs.yaml → Python → XML → Jenkins → Execution

The **pipeline structure**

Generates Jenkins job config dynamically

Dynamic value injected

```xml
<properties>
  <hudson.model.ParametersDefinitionProperty>
    <parameterDefinitions>
      <hudson.model.StringParameterDefinition>
        <name>BRANCH</name>
        <defaultValue>{{branch}}</defaultValue>
        <description>Branch to build</description>
      </hudson.model.StringParameterDefinition>
    </parameterDefinitions>
  </hudson.model.ParametersDefinitionProperty>
</properties>
```

jobs.yaml | job_template.xml

jenkins-template-engine > templates > job_template.xml > flow-definition

You, last week • V1:Add Initial V

```xml
<flow-definition plugin="workflow-job">
  <properties>
  </properties>

  <definition class="org.jenkinsci.plugins.workflow.cps.CpsFlowDefinition">
    <script>pipeline {
  agent any

  parameters {
    string(name: 'BRANCH', defaultValue: '{{branch}}')
  }

  stages {
    stage('Clone') {
      steps {
        git branch: params.BRANCH, url: '{{repo_url}}'
      }
    }

    stage('Install') {
      steps {
        bat '{{install_command}}'
      }
    }

    stage('Build') {
      steps {
        bat '{{build_command}}'
      }
    }

    stage('Test') {
      steps {
        bat '{{test_command}}'
      }
    }
  }
}</script>
    <sandbox>true</sandbox>
  </definition>

  <triggers/>
  <disabled>false</disabled>
</flow-definition>
```

PIPELINE STAGES

Each job includes:

1. Clone Repository
2. Install Dependencies
3. Build
4. Test

Commands come from YAML

Benefit:

Structured and standard CI workflow

jobs.yaml | job_template.xml

jenkins-template-engine > templates > job_template.xml > flow-definition

You, last week • V1:Add Initial V

```
<flow-definition plugin="workflow-job">
  <properties>
  </properties>

  <definition class="org.jenkinsci.plugins.workflow.cps.CpsFlowDefinition">
    <script>pipeline {
  agent any

  parameters {
    string(name: 'BRANCH', defaultValue: '{{branch}}')
  }

  stages {
    stage('Clone') {
      steps {
        git branch: params.BRANCH, url: '{{repo_url}}'
      }
    }

    stage('Install') {
      steps {
        bat '{{install_command}}'
      }
    }

    stage('Build') {
      steps {
        bat '{{build_command}}'
      }
    }

    stage('Test') {
      steps {
        bat '{{test_command}}'
      }
    }
  }
}</script>
      <sandbox>true</sandbox>
    </definition>

    <triggers/>
    <disabled>false</disabled>
  </flow-definition>
```

CRUMB HANDLING

Whenever we try to create or update a job using the Jenkins REST API, Jenkins rejects the request unless a valid CSRF token (called “crumb”) is included.

Step-by-Step Flow

- Fetch Crumb

  Python script sends a GET request to /crumbIssuer/api/json

  Jenkins returns: crumb value, crumbRequestField (header name)
- Add Crumb to Headers
- Send POST Request

  Used for:

  Creating job (/createItem)

  Updating job (/config.xml)
- Prevents unauthorized requests
- Required for all secure Jenkins API operations

Without crumb → request fails with 403 error

```python
def create_job(job_name, config_xml):
    crumb_url = f"{JENKINS_URL}/crumbIssuer/api/json"
    crumb_response = requests.get(crumb_url, auth=(USERNAME, API_TOKEN))

    if crumb_response.status_code != 200:
        print("Crumb fetch failed:", crumb_response.text)
        return

    crumb_data = crumb_response.json()
    crumb_header = {crumb_data['crumbRequestField']: crumb_data['crumb']}

    headers = {
        "Content-Type": "application/xml",
        **crumb_header
    }

    check_url = f"{JENKINS_URL}/job/{job_name}/api/json"
    check = requests.get(check_url, auth=(USERNAME, API_TOKEN))

    if check.status_code == 200:
        url = f"{JENKINS_URL}/job/{job_name}/config.xml"
        response = requests.post(
            url,
            data=config_xml,
            headers=headers,
            auth=(USERNAME, API_TOKEN)
        )
        action = "updated"
    else:
        url = f"{JENKINS_URL}/createItem?name={job_name}"
        response = requests.post(
            url,
            data=config_xml,
            headers=headers,
            auth=(USERNAME, API_TOKEN)
        )
        action = "created"
```

PARAMETERIZED PIPELINE

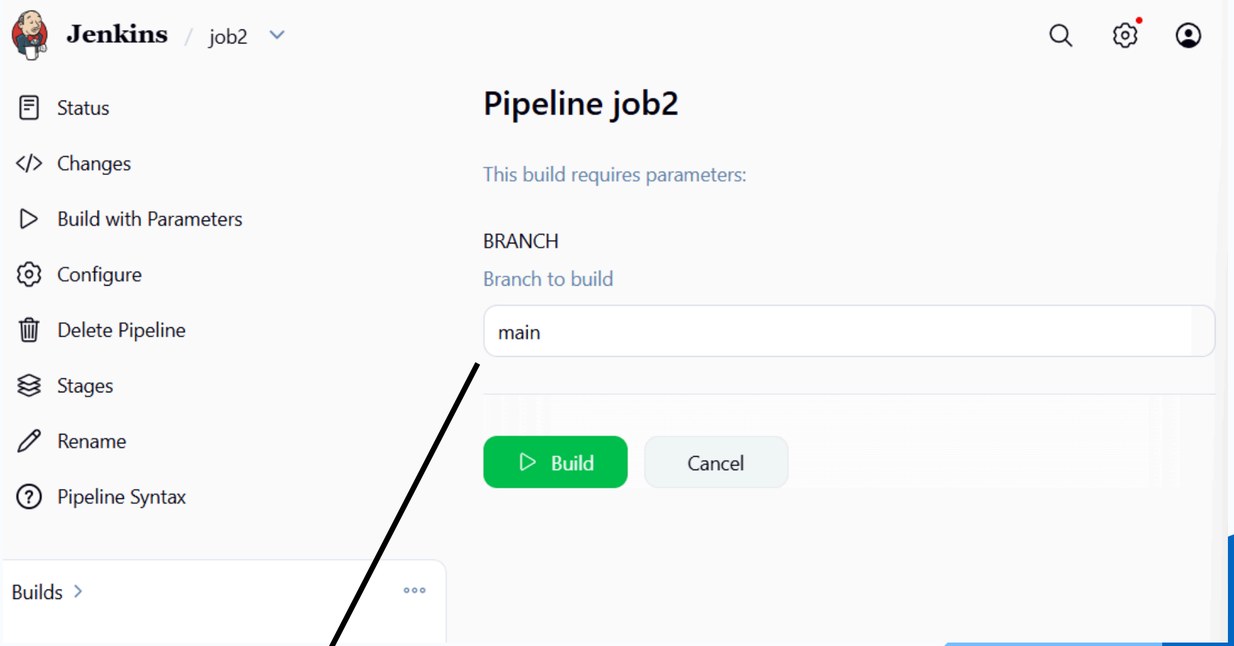
A parameterized pipeline allows dynamic input at runtime, instead of hardcoding values.

- User provides branch name at runtime
- Jenkins passes value
- Used in Git step:

git branch: params.BRANCH, url: repo_url

Importance

- Flexible pipeline
- Supports multiple environments (dev, staging, prod)
- No need to modify pipeline code again and again
- Dynamic CI/CD pipelines



Default branch is main

Untitled (Workspace)

EXPLORER

UNTITLED (WORKSP...
- Autonomous-Code-Reviewer-...
- Miscellaneous
- midterm_hostedPPT
- CarbonEmissionPredictionRN...
- jenkins-template-engine
  - configs
    - jobs.yaml
  - deliverables
    - final-report.pdf U
  - docs
    - screenshots
    - design-document.md U
    - project-plan.md U
    - user-guide.md U
  - pipelines
    - Jenkinsfile U
  - presentations
    - demo-script.md U
  - src\scripts
    - generate_jobs.py 3, U
  - templates
    - job_template.xml
  - .gitignore U
  - README.md
- data-drift-prediction-in-ml-models

OUTLINE

TIMELINE

job_template.xml | README.md | generate_jobs.py 3, U | Jenkinsfile U | jobs.yaml

jenkins-template-engine > pipelines > Jenkinsfile

```
pipeline {
    agent any

    parameters {
        string(name: 'BRANCH', defaultValue: 'main')
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: params.BRANCH, url: 'https://github.com/Anwesha-code/jenkins-job-template-engin
            }
        }

        stage('Build') {
            steps {
                bat 'echo Building project'
            }
        }
    }
}
```

PROBLEMS 3 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

powershell - scripts

```
PS C:\Projects\jenkins-template-engine\src\scripts> py generate_jobs.py
      }
    }

    stage('Build') {
      steps {
        bat 'echo Building Job 2'
      }
    }

    stage('Test') {
      steps {
        bat 'echo Running tests'
      }
    }
  }
}</script>
    <sandbox>true</sandbox>
  </definition>

  <triggers/>
  <disabled>false</disabled>
</flow-definition>
job2 created successfully
```

All +

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓		job1	18 min #1	N/A	14 sec
✗		job2	N/A	3 min 8 sec #1	7.6 sec

Icon: S M L

Jenkins / job2 / #1

```
[Pipeline] stage
[Pipeline] { (Install)
Stage "Install" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
Stage "Build" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
Stage "Test" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: Couldn't find any revision to build. Verify the repository and branch configuration for this job.
Finished: FAILURE
```

- Git step fails
- Jenkins cannot checkout code
- Pipeline stops

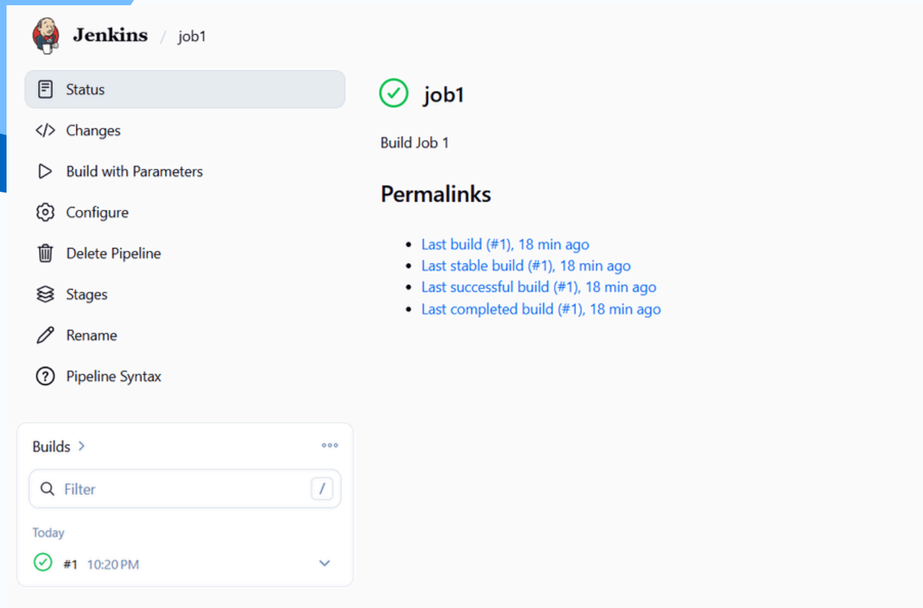
job 1 → SUCCESS

- Valid Branch
- Pipeline completed successfully

job 2 → FAILURE

- Invalid branch
- Branch passed does not exist in Git repository
- Pipeline failed early

CI pipelines depend on correct Git configuration




```
Started by user Anwesha Singh
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\job1
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Clone)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
 > git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\job1\.git # timeout=10
Fetching changes from the remote Git repository
 > git.exe config remote.origin.url https://github.com/Anwesha-code/jenkins-job-template-engine.git # timeout=10
Fetching upstream changes from https://github.com/Anwesha-code/jenkins-job-template-engine.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.52.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/Anwesha-code/jenkins-job-template-engine.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe rev-parse "refs/remotes/origin/dev^{commit}" # timeout=10
Checking out Revision 9ed1e746c4eec0294d828e5a7a003e534b414b06 (refs/remotes/origin/dev)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f 9ed1e746c4eec0294d828e5a7a003e534b414b06 # timeout=10
 > git.exe branch -a -v --no-abbrev # timeout=10
 > git.exe branch -D dev # timeout=10
 > git.exe checkout -b dev 9ed1e746c4eec0294d828e5a7a003e534b414b06 # timeout=10
Commit message: " V1.2.0: Add working parameterized job template"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Install)
[Pipeline] bat
```

```
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Install)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\job1>echo Installing dependencies
Installing dependencies
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\job1>echo Building Job 1
Building Job 1
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\job1>echo Running tests
Running tests
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Pipeline Flow

1. Clone → success
2. Install → success
3. Build → success
4. Test → success

STAGES

1. **Clone Stage**
   - Git repository fetched successfully
   - Correct branch checked out
2. **Install Stage**
   - echo Installing dependencies
3. **Build Stage**
   - echo Building Job 1
4. **Test Stage**
   - echo Running tests

Commands executed exactly as defined in YAML

FAIL-FAST

Fail-fast means:
Stop execution immediately when an error occurs
example:
- Clone stage fails
- Install, Build, Test → skipped

Benefits:
Saves Time: No unnecessary execution
Saves Resources: CPU, memory, build agents
Easier Debugging: Error identified at source

THANKYOU

Questions?